

SYSTEM DESIGN

STEP 2: ESTIMATE SCALE & BOTTLENECKS

Estimate System Scale

➤ Daily Load Estimates:

- 500M total users | 200M Daily active users
- ~1B tweets/day | ~300M media uploads
- ~1B likes, 500M replies, 250M retweets
- 2B+ feed requests/day

Scalability

Ability of a system to handle an increasing amount of work, or its potential to accommodate growth

It ensures performance reliability and availability under growing load

Why do system need to scale?



User base growth



More users over time



Increase data volume



More data stored
and processed



Peak events



Sudden spikes in
traffic and demand



Avoid service degradation
or downtime



Ensure high availability
and excellent user experience

Common challenges in scaling



- Latency
- Bottlenecks
- Downtime
- Cost

Types of scaling

- Vertical scaling
- Horizontal scaling
- Diagonal scaling

Vertical scaling (Scale Up)

Increasing the resources of an existing server to handle more traffic and workload.

Resources That Can Be Increased

- CPU
- RAM
- Storage
- Network Capacity



VERTICAL SCALING

ADVANTAGES

BEFORE



Smaller Server
Lower Resources



AFTER

More Resources
(CPU, RAM, Storage)



1. Simple to Implement

Upgrade the existing server resources like CPU, RAM or Storage.



2. Easy Management

Easier to manage since there is only one server to maintain.



3. No Load Balancer Required

No need for a load balancer as there is only one server.



4. Suitable for Small Applications

Cost-effective and ideal for small applications with moderate traffic.



Vertical Scaling is cost-effective and effective for applications with growing resource needs.

VERTICAL SCALING

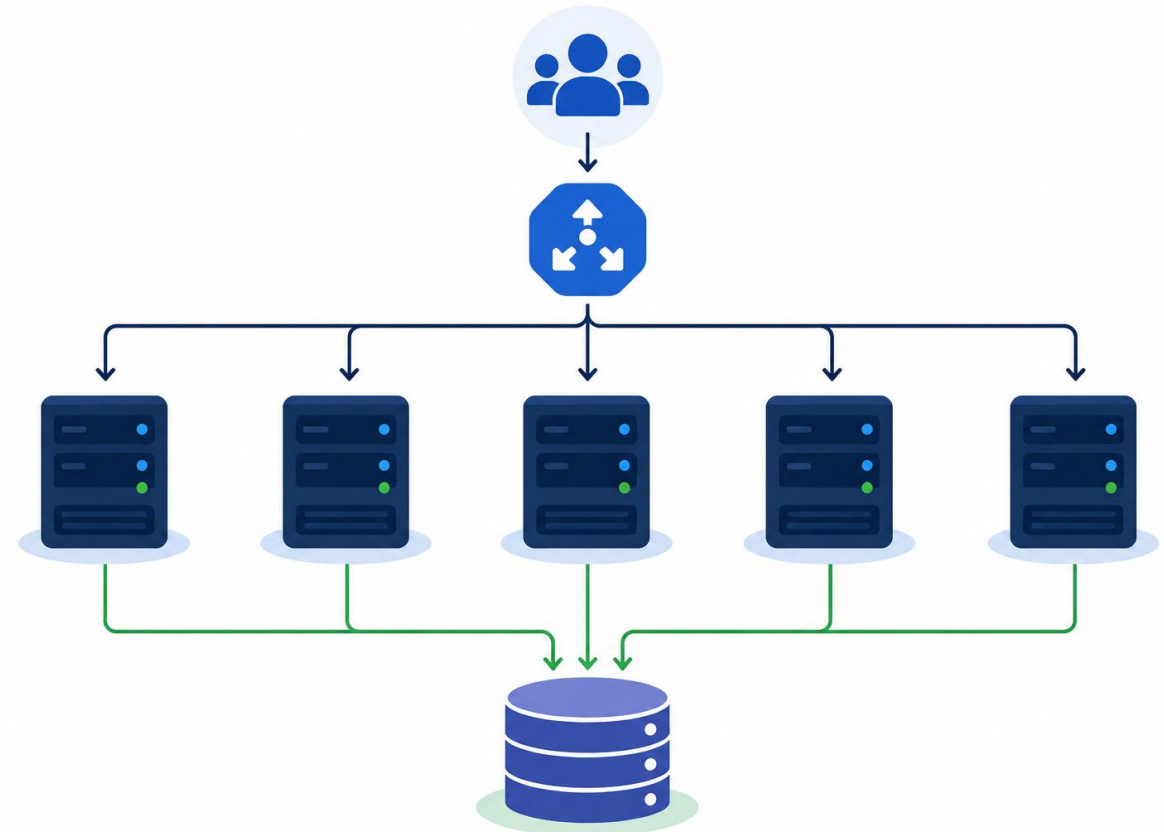
DISADVANTAGES



Vertical scaling is suitable for small applications, but it has several limitations for high growth and high availability needs.

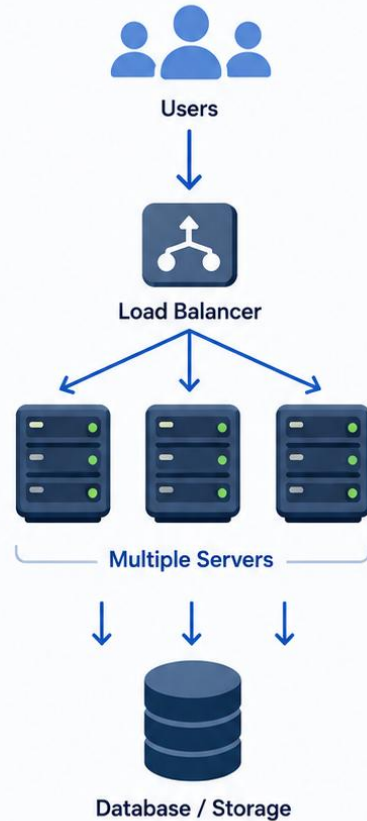
Horizontal Scaling (Scale Out)

Increasing system capacity by **adding more servers** instead of upgrading a single server.



— HORIZONTAL SCALING —

ADVANTAGES



1. High Scalability

Easily scale by adding more servers to handle increased traffic and growing demand.



2. High Availability

If a server fails, other servers continue to serve requests, ensuring minimal downtime.



3. Fault Tolerance

Failures of individual servers do not affect the overall system. Traffic is automatically rerouted.



4. Better Performance

Distributes load across multiple servers, reducing response time and improving user experience.



5. Cost Effective

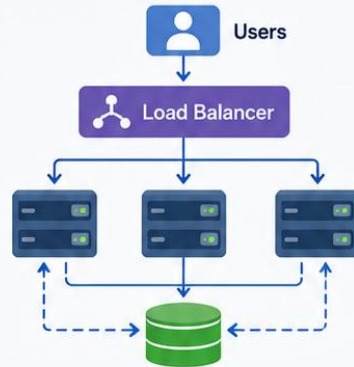
Uses commodity servers which are more affordable than high-end powerful servers.



Horizontal scaling adds more machines to scale out the system, providing better scalability, availability, reliability and cost efficiency.

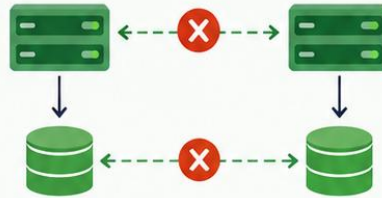
HORIZONTAL SCALING

DISADVANTAGES



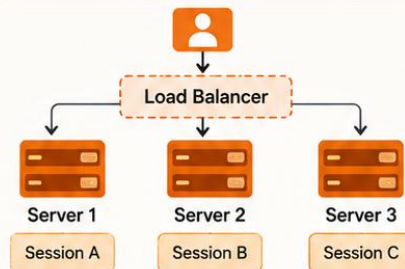
1 More Complex Architecture

Requires multiple servers, load balancers, shared databases, and additional infrastructure, making design, deployment and maintenance more complex.



2 Data Consistency Challenges

Keeping data consistent across multiple servers and databases can be difficult, especially in distributed systems.



3 Session Management Issues

Managing user sessions across multiple servers is challenging. Additional solutions like sticky sessions or centralized session stores are needed.



While horizontal scaling offers great benefits, it introduces complexity in system design, data consistency, and session management.

Diagonal Scaling

Diagonal Scaling is a combination of **Vertical Scaling (Scale Up)** and **Horizontal Scaling (Scale Out)**.

The system first increases the capacity of existing servers (vertical scaling), and when the server reaches its limit, additional servers are added (horizontal scaling).

ADVANTAGES OF DIAGONAL SCALING



1 Efficient Resource Utilization

Balances CPU, memory, and storage needs by scaling compute and storage simultaneously, reducing resource bottlenecks.



2 Cost Effective

Adds only the necessary resources in the right proportion, avoiding over-provisioning and unnecessary costs.



3 Flexible Growth

Easily adapts to changing workloads by scaling compute and storage together as per application demands.



4 Better Scalability

Supports both vertical and horizontal scaling, providing better performance, availability, and the ability to handle higher loads.



Diagonal scaling combines the best of vertical and horizontal scaling, delivering optimal performance, efficiency, and growth.

DISADVANTAGES OF DIAGONAL SCALING



Start Small

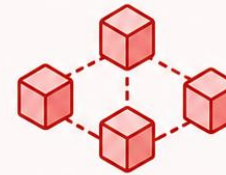


Scale Up and Out Together



1 More Complex

Involves both vertical and horizontal scaling, which increases architectural complexity, planning, deployment, and ongoing management.



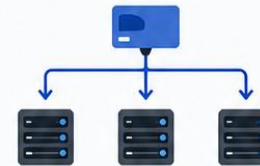
2 Hardware Limit Still Exist

Vertical scaling is still limited by the maximum capacity of a single machine (CPU, RAM, storage).



3 Requires Load Balancing

Needs load balancers to distribute traffic across multiple servers, adding extra infrastructure, configuration, and cost.



Diagonal scaling offers flexibility and better performance, but it comes with increased complexity, hardware limits, and the need for load balancing.

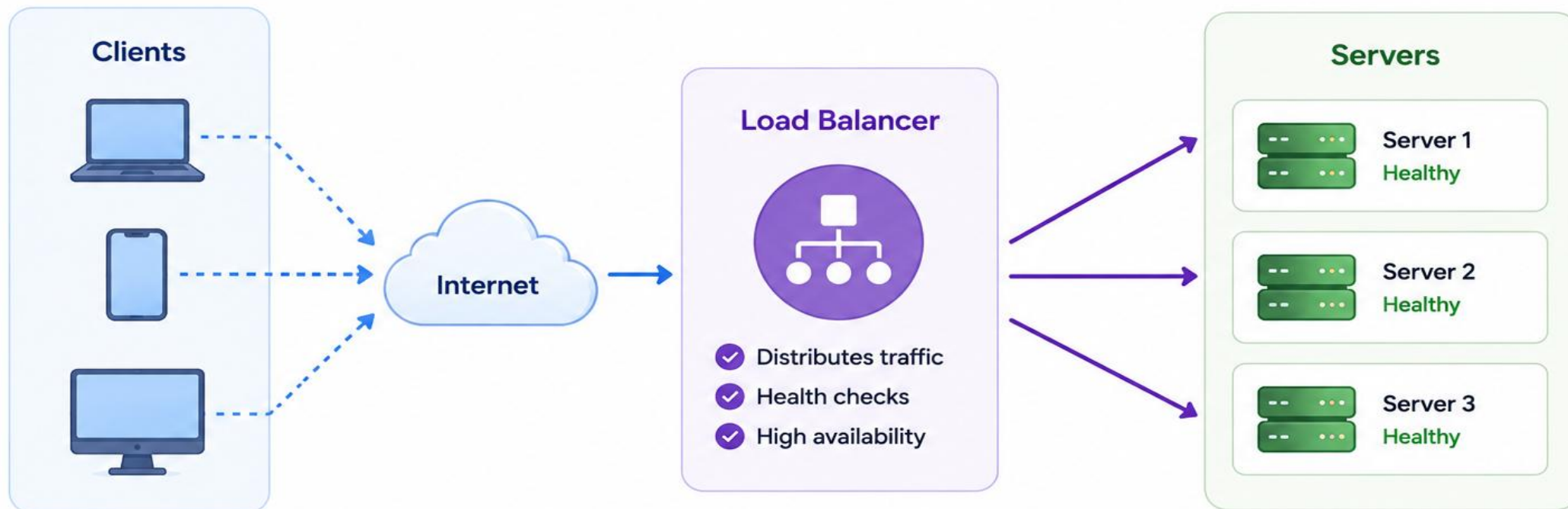
Auto Scaling

Auto Scaling is the automatic process of increasing or decreasing the number of servers/resources based on the application's current workload and traffic.

Instead of manually adding or removing servers, the system automatically adjusts capacity.

Load Balancing

Distributes incoming traffic across multiple servers to ensure no single server is overwhelmed.



Benefits:



High Availability

Scalability

Better Performance

Fault Tolerance

Efficient Resource Utilization

Two types of load balancers

Based on Layer

- Layer 4 Load Balancer
(Transport Layer)
- Layer 7 Load Balancer
(Application Layer)

Based on Deployment

- Hardware Load Balancer
- Software Load Balancer
- Cloud Load Balancer

Layer 4 (Transport Layer) Load Balancer

- Operates at the TCP/UDP level.
- Routes traffic based on IP addresses and port numbers.
- Faster because it doesn't inspect the request content.

Examples:

- TCP Load Balancer
- Network Load Balancer (NLB)

Layer 7 (Application Layer) Load Balancer

- Operates at the HTTP/HTTPS level.
- Can inspect URLs, headers, cookies, and request content.
- Enables intelligent routing.

Examples:

- Route /images to Image Server
- Route /payments to Payment Server

Hardware Load Balancer

- Physical appliance installed in a data center.
- High performance and reliability.
- Expensive and less flexible.

Examples:

- F5 Networks BIG-IP
- Cisco load balancing solutions

Software Load Balancer

- Runs as software on servers or VMs.
- Cost-effective and easy to scale.
- Widely used in modern cloud environments.

Examples:

- Nginx
- HAProxy

Cloud Load Balancer

- Managed by cloud providers.
- No infrastructure management required.
- Automatically scales with traffic.

Examples:

- AWS Elastic Load Balancing
- Google Cloud Load Balancing
- Azure Load Balancer

Load Balancing Strategies



Static Load Balancing:

In **Static Load Balancing**, the distribution of requests is decided using predefined rules and does not consider the current state or workload of the servers.

Characteristics

- Simple to implement.
- Low overhead.
- Assumes all servers have similar capacity.
- Does not adapt to changing server loads.

Dynamic Load Balancing

- In **Dynamic Load Balancing**, the load balancer continuously monitors server health and workload, distributing requests based on real-time conditions.

Characteristics

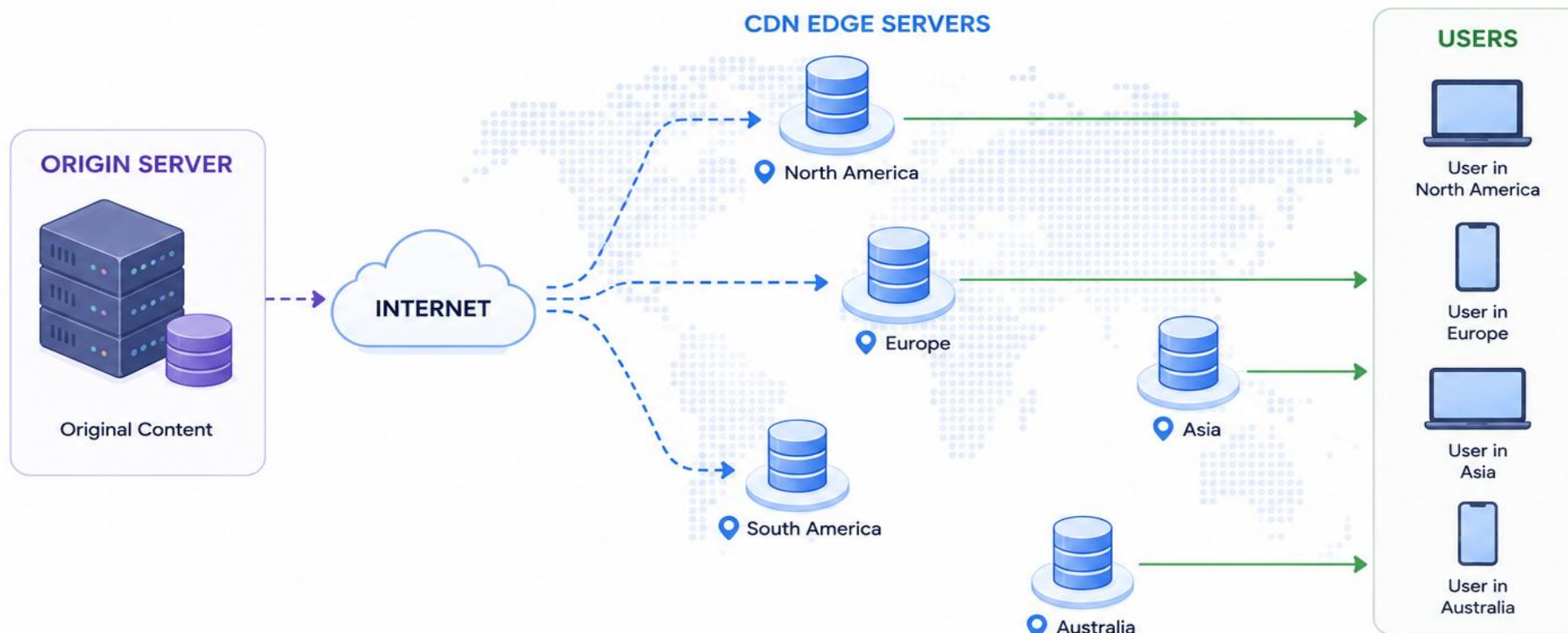
- Adapts to changing traffic patterns.
- Considers metrics such as CPU usage, memory utilization, response time, and active connections.
- More efficient in complex environments.

➤ Media Storage & Delivery

- $\sim 300\text{M media uploads/day} = 100\text{-}500\text{TB/day}$
- Must Support
 - Upload APIs
 - Object storage
 - CDN-backed delivery

CDN (Content Delivery Network)

Delivers content to users from the nearest server location for faster performance and reliability.



KEY BENEFITS



Faster
Performance



High
Availability



Reduced
Bandwidth Costs



Global
Reach



Better User
Experience

➤ Read > Write :80% traffic is read - heavy

Caching

Caching is the process of storing frequently accessed data in a temporary high-speed storage layer (cache) so that future requests can be served faster.

Instead of repeatedly fetching data from a slow source (database, disk, or external service), the application retrieves it from the cache.

Without caching

User Request



Application



Database



Return Data

Types of Caching

- Client-side Cache – stored in user's browser
- CDN cache- Stored on edge servers close to users
- Application Cache- Stored between the application and database
- Database Cache- Database itself caches frequently accessed queries and data pages

Identifying System Bottlenecks

Timeline = Fan-out at Scale

- Celebrity posts 1 tweet → needs to be visible to 1M followers

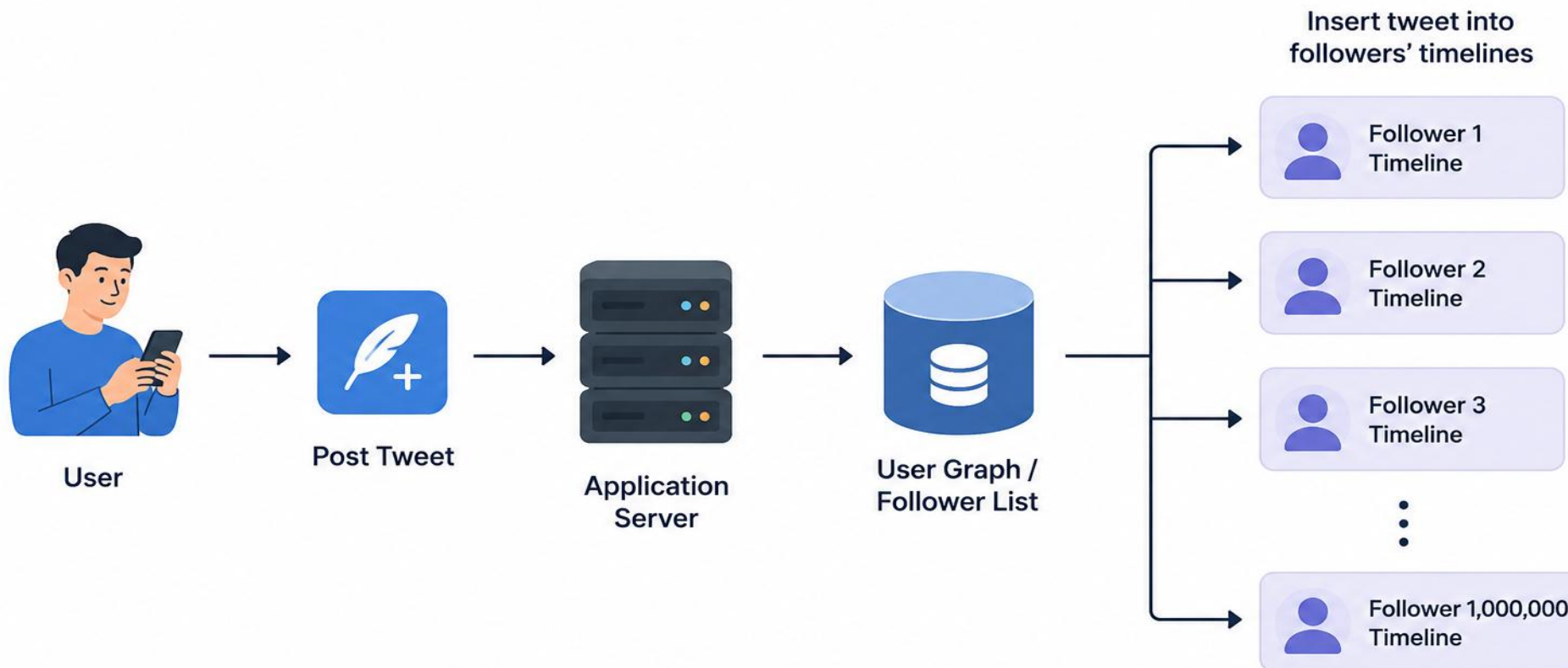
Fan-out = One write operation triggering many downstream updates.

Two Fan-out Models

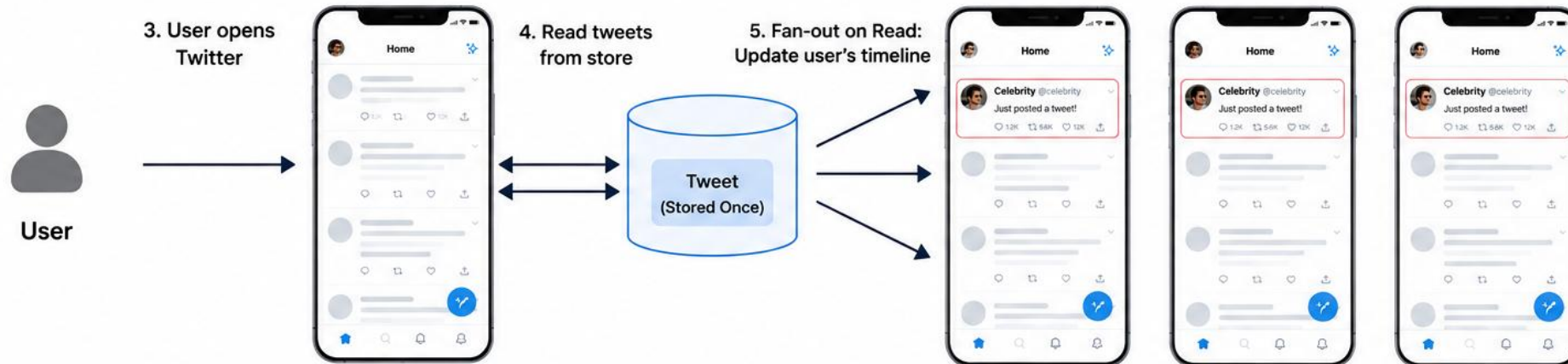
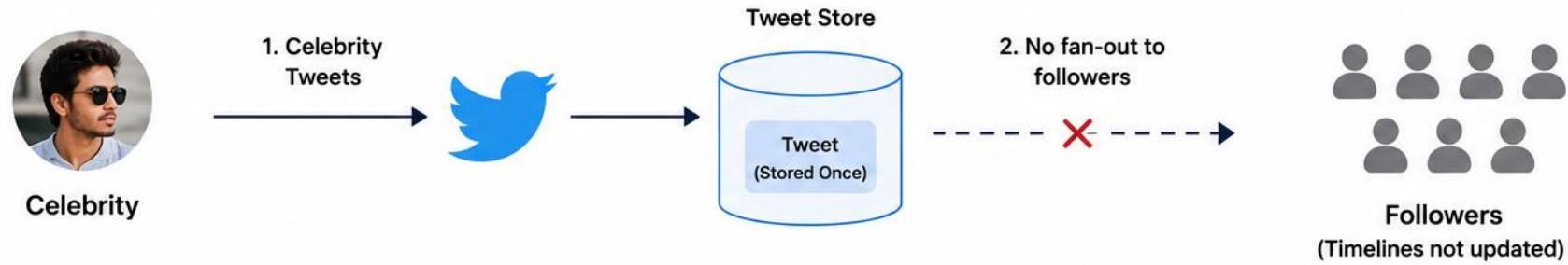
- Fan-out on Read
- Fan-out on Write

Fan-out on Write

When a user posts a tweet

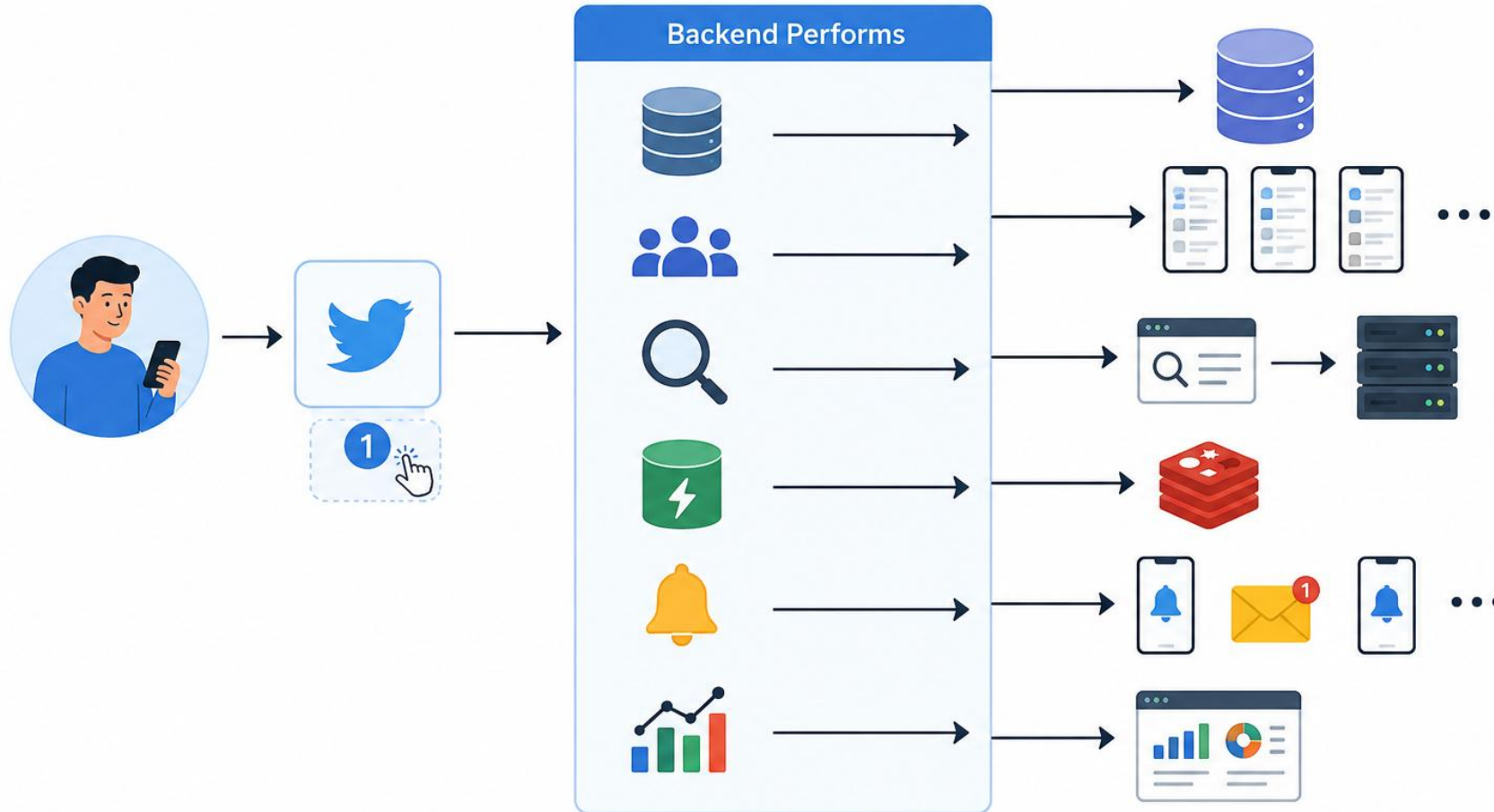


Fan-out on Read



Write Amplification

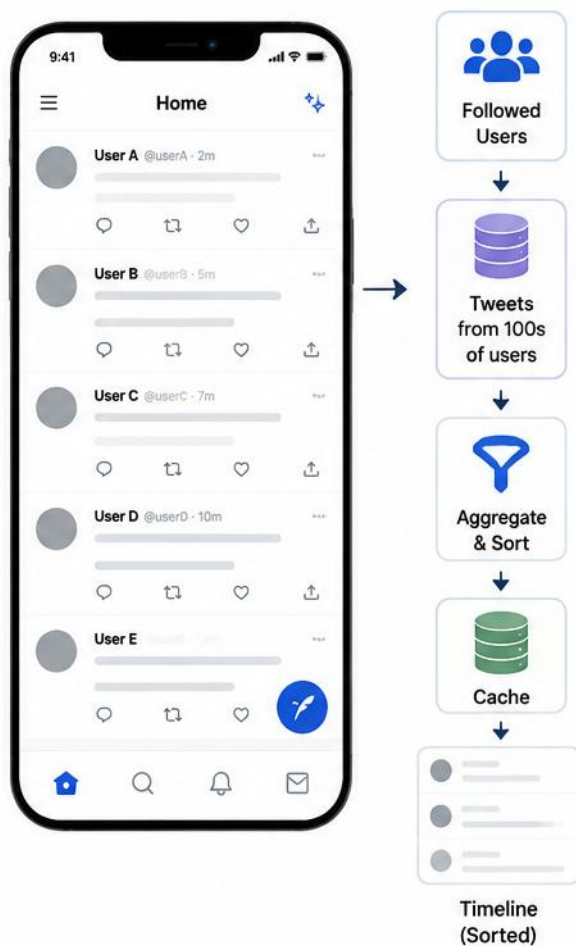
Example 1: Posting a Tweet



Read Heavy Patterns

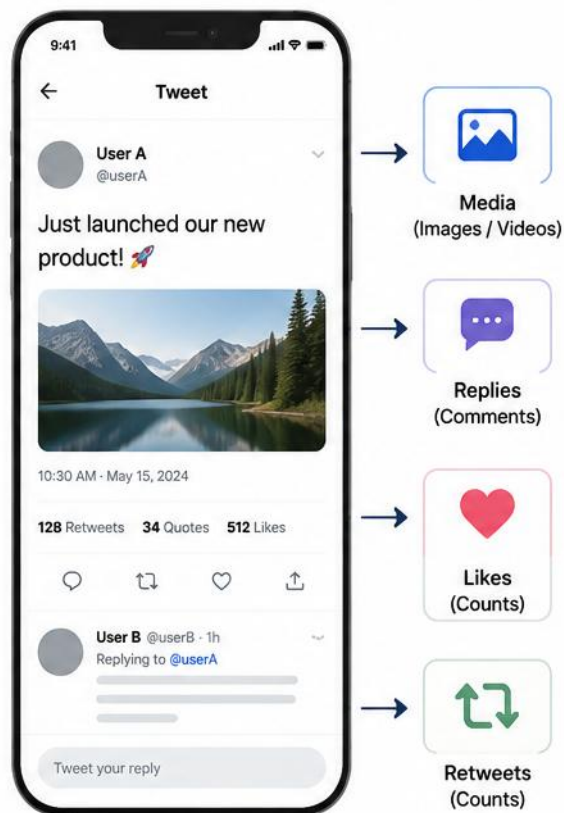
1 Opening the timeline

Aggregates and sorts tweets from 100s of followed users



2 Viewing a tweet

Triggers fetches for media, replies, likes, retweets



3 Scrolling

Causes pagination, cache lookups and lazy load

